部署管理API設計書

メタデータ

- **作成日**: 2026-02-13
- **バージョン**: 1.0.0
- **最終更新**: 2026-02-13
- **担当**: Development Team
- **実装状況**: ✅ 実装済み（100%）

概要

このドキュメントは、組織内の部署マスタを管理するためのAPI仕様を定義します。

主な機能

- 部署一覧の取得（検索・フィルタリング・ソート対応）
- 部署詳細の取得
- 部署の新規作成
- 部署情報の更新
- 部署のアクティブ状態変更
- CSVによる一括ダウンロード
- CSVによる一括アップロード（バッチ処理）

対象ユーザー

- システム管理者（System Admin）: 全機能へのアクセス
- データ管理者（Data Admin）: 全機能へのアクセス

📋 API一覧

No	HTTPメソッド	エンドポイント	説明	実装状態	権限
1	GET	/api/v1/systemManagement/department	部署一覧取得	✅ 実装済み	System Admin, Data Admin
2	GET	/api/v1/systemManagement/department/download-csv	部署CSVダウンロード	✅ 実装済み	System Admin, Data Admin
3	POST	/api/v1/systemManagement/department/upload-csv	部署CSVアップロード	✅ 実装済み	System Admin, Data Admin
4	GET	/api/v1/systemManagement/department/:departmentCd	部署詳細取得	✅ 実装済み	System Admin, Data Admin
5	POST	/api/v1/systemManagement/department	部署新規作成	✅ 実装済み	System Admin, Data Admin
6	PUT	/api/v1/systemManagement/department/:departmentCd	部署更新	✅ 実装済み	System Admin, Data Admin
7	PATCH	/api/v1/systemManagement/department/:departmentCd/status	部署アクティブ状態更新	✅ 実装済み	System Admin, Data Admin

---

1. 部署一覧取得API

概要

組織内の部署一覧を取得します。検索、フィルタリング、ソート機能に対応します。

エンドポイント

```
GET /api/v1/systemManagement/department
```

HTTPメソッド

GET

認証要件

- JWT認証必須
- ロール: System Admin または Data Admin
- テナント分離必須

ミドルウェアチェーン

```
router.get('/department',
  authMiddleware,              // JWT認証（organizationIdを取得）
  handleValidationErrors,      // バリデーションエラーハンドリング
  departmentController.getDepartments
);
```

リクエスト

クエリパラメータ

パラメータ	型	必須	デフォルト	説明
search	string	×	-	部署CDまたは部署名で検索
is_active	boolean	×	-	アクティブ状態でフィルタ（true/false）
sort_by	string	×	-	ソート項目（department_cd, department_name, update_at）
sort_order	string	×	asc	ソート順（asc/desc）

リクエスト例

```
GET /api/v1/systemManagement/department?search=営業
&is_active=true&sort_by=department_name&sort_order=asc
```

レスポンス

成功時（200 OK）

```
{
  "success": true,
  "data": [
    {
      "departmentId": "DEPT001",
      "displayName": "営業部",
      "isActive": true,
      "createdAt": "2026-01-01T00:00:00Z",
      "updatedAt": "2026-02-13T10:00:00Z"
    },
    {
      "departmentId": "DEPT002",
      "displayName": "開発部",
      "isActive": true,
      "createdAt": "2026-01-01T00:00:00Z",
      "updatedAt": "2026-02-13T10:00:00Z"
    }
  ],
  "meta": {
    "timestamp": "2026-02-13T15:00:00Z",
    "requestId": "req_abc123"
  }
}
```


エラー時（400 Bad Request）


```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Validation failed",
    "details": [
      {
        "field": "sort_order",
        "message": "sort_order must be either \"asc\" or \"desc\"",
        "code": "INVALID_VALUE"
      }
    ]
  },
  "meta": {
    "timestamp": "2026-02-13T15:00:00Z",
    "requestId": "req_abc123"
```

```
  }
}
```

ビジネスロジック

1. **認証チェック**
   - JWT認証でorganizationIdを取得
   - System AdminまたはData Adminの権限を確認
2. **フィルタリング**
   - searchパラメータ: 部署CDまたは部署名に対して部分一致検索
   - is_activeパラメータ: アクティブ状態でフィルタリング
3. **ソート**
   - sort_byで指定されたフィールドでソート
   - sort_orderで昇順/降順を制御
4. **データ取得**
   - organization_idでフィルタ（マルチテナント分離）
   - フィルタ・ソート条件を適用してデータ取得

バリデーション

必須チェック

- [ ] 認証トークンが有効
- [ ] organizationIdが取得できる

値チェック

- [ ] searchは文字列（指定時）
- [ ] is_activeはboolean（指定時）
- [ ] sort_byは許可された値（department_cd, department_name, update_at）

- [ ] sort_orderは"asc"または"desc"

データベーステーブル

使用テーブル

1. **department**（主テーブル）
    - organization_id, department_cd, department_name, is_active
    - create_user_id, create_at, update_user_id, update_at

SQL例

```
SELECT
  department_cd,
  department_name,
  is_active,
  create_at,
  update_at
FROM department
WHERE organization_id = $1
  AND ($2::text IS NULL OR department_cd ILIKE '%' || $2 || '%' OR department_name ILIKE '%'
|| $2 || '%')
  AND ($3::boolean IS NULL OR is_active = $3)
ORDER BY
  CASE WHEN $4 = 'department_cd' THEN department_cd END ASC,
  CASE WHEN $4 = 'department_name' THEN department_name END ASC,
  CASE WHEN $4 = 'update_at' THEN update_at END DESC;
```

セキュリティ

マルチテナント分離

- [ ] organization_id フィルタ必須
- [ ] 自組織の部署のみ取得可能

認証・認可

- [ ] JWT認証必須
- [ ] System AdminまたはData Adminの権限が必要

入力検証

- [ ] SQLインジェクション対策（Parameterized Queries）

パフォーマンス

- インデックス: department (organization_id, is_active)
- インデックス: department (organization_id, department_name)
- トランザクション: 不要（読み取り専用）
- キャッシュ: 不要（リアルタイム性重視）

---

2. 部署CSVダウンロードAPI

概要

部署一覧をCSV形式でダウンロードします。フィルタ条件を適用できます。

エンドポイント

```
GET /api/v1/systemManagement/department/download-csv
```

HTTPメソッド

GET

認証要件

- JWT認証必須

- ロール: System Admin または Data Admin
- テナント分離必須

ミドルウェアチェーン

```
router.get('/department/download-csv',
  authMiddleware,
  departmentController.downloadDepartmentCsv
);
```

リクエスト

クエリパラメータ

パラメータ	型	必須	デフォルト	説明
search	string	×	-	部署CDまたは部署名で検索
is_active	boolean	×	-	アクティブ状態でフィルタ

リクエスト例

```
GET /api/v1/systemManagement/department/download-csv?is_active=true
```

レスポンス

成功時（200 OK）

Content-Type: `text/csv; charset=utf-8`

ファイル名: `departments_YYYYMMDDhhmmss.csv`

CSV形式 (UTF-8 BOM付き):

```
department_cd,department_name,update_at,is_active
DEPT001,営業部,2026-02-13T10:00:00Z,true
DEPT002,開発部,2026-02-13T10:00:00Z,true
DEPT003,管理部,2026-02-13T10:00:00Z,false
```

ビジネスロジック

1. **データ取得**
   - getDepartments()と同じフィルタ条件を適用
   - organization_idでフィルタ
2. **CSV生成**
   - Papa.unparseを使用してCSV生成
   - UTF-8 BOMを付与（Excel対応）
3. **ファイル名生成**
   - `departments_YYYYMMDDhhmmss.csv` 形式
   - タイムスタンプを含める
4. **レスポンス**
   - Content-Typeヘッダーを設定
   - Content-Dispositionヘッダーでファイル名を指定

バリデーション

- [ ] 認証トークンが有効
- [ ] organizationIdが取得できる
- [ ] フィルタパラメータが有効（指定時）

データベーステーブル

- 部署一覧取得APIと同じ

セキュリティ

- [ ] organization_id フィルタ必須
- [ ] JWT認証必須
- [ ] 適切な権限チェック

パフォーマンス

- 大量データの場合はストリーミング処理を検討
- インデックス活用でクエリ高速化

---

3. 部署CSVアップロードAPI

概要

CSV形式で部署データを一括登録・更新します。

エンドポイント

```
POST /api/v1/systemManagement/department/upload-csv
```

HTTPメソッド

POST

認証要件

- JWT認証必須
- ロール: System Admin または Data Admin
- テナント分離必須

ミドルウェアチェーン

```
router.post('/department/upload-csv',
  authMiddleware,
  upload.single('file'),      // Multerミドルウェア
  departmentController.uploadDepartmentCsv
);
```

リクエスト

マルチパートフォームデータ

Content-Type: `multipart/form-data`

フィールド	型	必須	説明
file	File	○	CSVファイル

CSV形式

```
department_cd,department_name,is_active
DEPT001,営業部,true
DEPT002,開発部,true
DEPT003,管理部,false
```

カラム定義:

- `department_cd` : 部署CD（必須、50文字以内）
- `department_name` : 部署名（必須、200文字以内）
- `is_active` : アクティブ状態（必須、true/false）

レスポンス

成功時（200 OK）

```
{
  "success": true,
  "data": {
    "created": 2,
    "updated": 1,
    "errors": []
  },
  "meta": {
    "timestamp": "2026-02-13T15:00:00Z",
    "requestId": "req_abc123"
  }
}
```

エラー時（400 Bad Request）

```
{
  "success": false,
  "error": {
    "code": "FILE_REQUIRED",
    "message": "CSVファイルが必要です"
  },
  "meta": {
    "timestamp": "2026-02-13T15:00:00Z",
    "requestId": "req_abc123"
  }
}
```

バリデーションエラー（200 OK with errors）

```
{
  "success": true,
  "data": {
    "created": 1,
    "updated": 0,
    "errors": [
      {
        "row": 2,
        "departmentCd": "DEPT999",
        "error": "部署名が必須です"
      }
    ]
```

```
  },
  "meta": {
    "timestamp": "2026-02-13T15:00:00Z",
    "requestId": "req_abc123"
  }
}
```

ビジネスロジック

1. **ファイル検証**
   - CSVファイルの存在確認
   - ファイルサイズチェック（最大10MB）
   - MIMEタイプチェック（text/csv）
2. **CSVパース**
   - Papa.parseを使用
   - ヘッダー行の検証
   - 各行のバリデーション
3. **データ処理**
   - トランザクション開始
   - 既存データの確認（department_cd）
   - 新規作成または更新
   - エラー発生時はロールバック
4. **結果集計**
   - 作成件数、更新件数、エラー件数を集計
   - エラー詳細を返却

バリデーション

ファイルチェック

- [ ] ファイルが存在する

- [ ] ファイルサイズが10MB以下
- [ ] MIMEタイプがtext/csv

データチェック（各行）

- [ ] department_cdが存在する
- [ ] department_cdが50文字以内
- [ ] department_nameが存在する
- [ ] department_nameが200文字以内
- [ ] is_activeがboolean（true/false）

データベーステーブル

使用テーブル

1. **department**（主テーブル）

SQL例

```
-- トランザクション開始
BEGIN;

-- 既存データ確認
SELECT department_cd FROM department
WHERE organization_id = $1 AND department_cd = $2;

-- 新規作成
INSERT INTO department (
  organization_id, department_cd, department_name, is_active,
  create_user_id, create_at, update_user_id, update_at
) VALUES ($1, $2, $3, $4, $5, NOW(), $5, NOW());

-- 更新
UPDATE department
SET
  department_name = $3,
  is_active = $4,
  update_user_id = $5,
  update_at = NOW()
WHERE organization_id = $1 AND department_cd = $2;
```

```
COMMIT;
```

セキュリティ

マルチテナント分離

- [ ] organization_id フィルタ必須
- [ ] 自組織のデータのみ操作可能

ファイルアップロード対策

- [ ] ファイルサイズ制限（10MB）
- [ ] 許可されたMIMEタイプのみ
- [ ] ファイル内容のバリデーション

入力検証

- [ ] SQLインジェクション対策
- [ ] CSVインジェクション対策

パフォーマンス

- バッチ処理でINSERT/UPDATE
- トランザクション使用
- エラー時は全ロールバック

---

4. 部署詳細取得API

概要

部署CDを指定して、部署の詳細情報を取得します。

エンドポイント

```
GET /api/v1/systemManagement/department/:departmentCd
```

HTTPメソッド

GET

認証要件

- JWT認証必須
- ロール: System Admin または Data Admin
- テナント分離必須

ミドルウェアチェーン

```
router.get('/department/:departmentCd',
  authMiddleware,
  [
    param('departmentCd')
      .notEmpty()
      .withMessage('departmentCd is required')
      .isString()
      .withMessage('departmentCd must be a string'),
  ],
  handleValidationErrors,
  departmentController.getDepartmentById
);
```

リクエスト

パスパラメータ

パラメータ	型	必須	説明
departmentCd	string	○	部署CD

リクエスト例

```
GET /api/v1/systemManagement/department/DEPT001
```

レスポンス

成功時（200 OK）

```
{
  "success": true,
  "data": {
    "departmentId": "DEPT001",
    "displayName": "営業部",
    "isActive": true,
    "createdAt": "2026-01-01T00:00:00Z",
    "updatedAt": "2026-02-13T10:00:00Z"
  },
  "meta": {
    "timestamp": "2026-02-13T15:00:00Z",
    "requestId": "req_abc123"
  }
}
```

エラー時（404 Not Found）

```
{
  "success": false,
  "error": {
    "code": "DEPARTMENT_NOT_FOUND",
    "message": "Department not found"
  },
  "meta": {
    "timestamp": "2026-02-13T15:00:00Z",
    "requestId": "req_abc123"
  }
}
```

ビジネスロジック

1. **データ取得**
    - department_cdとorganization_idで部署を検索
    - 該当データが存在しない場合は404エラー

バリデーション

- [ ] departmentCdが空でない
- [ ] departmentCdが文字列

データベーステーブル

SQL例

```
SELECT
  department_cd,
  department_name,
  is_active,
  create_at,
  update_at
FROM department
WHERE organization_id = $1 AND department_cd = $2;
```

セキュリティ

- [ ] organization_id フィルタ必須
- [ ] JWT認証必須

パフォーマンス

- プライマリキー検索（高速）
- トランザクション不要

5. 部署新規作成API

概要

新しい部署を作成します。

エンドポイント

```
POST /api/v1/systemManagement/department
```

HTTPメソッド

POST

認証要件

- JWT認証必須
- ロール: System Admin または Data Admin
- テナント分離必須

ミドルウェアチェーン

```
router.post('/department',
  authMiddleware,
  [
    body('departmentCd')
      .notEmpty()
      .withMessage('departmentCd is required')
      .isString()
      .withMessage('departmentCd must be a string'),
    body('departmentName')
      .notEmpty()
      .withMessage('departmentName is required')
      .isString()
      .withMessage('departmentName must be a string'),
    body('isActive')
```

```
      .optional()
      .isBoolean()
      .withMessage('isActive must be a boolean'),
  ],
  handleValidationErrors,
  departmentController.createDepartment
);
```

リクエスト

リクエストボディ

```
{
  "departmentCd": "DEPT003",
  "departmentName": "管理部",
  "isActive": true
}
```

パラメータ	型	必須	デフォルト	説明
departmentCd	string	○	-	部署CD（50文字以内）
departmentName	string	○	-	部署名（200文字以内）
isActive	boolean	×	true	アクティブ状態

レスポンス

成功時（201 Created）

```
{
  "success": true,
  "data": {
    "departmentId": "DEPT003",
    "displayName": "管理部",
    "isActive": true,
    "createdAt": "2026-02-13T15:00:00Z",
    "updatedAt": "2026-02-13T15:00:00Z"
  },
  "meta": {
```

```
    "timestamp": "2026-02-13T15:00:00Z",
    "requestId": "req_abc123"
  }
}
```

エラー時（409 Conflict）

```
{
  "success": false,
  "error": {
    "code": "DUPLICATE_DEPARTMENT_CD",
    "message": "Department code already exists"
  },
  "meta": {
    "timestamp": "2026-02-13T15:00:00Z",
    "requestId": "req_abc123"
  }
}
```

ビジネスロジック

1. **重複チェック**
   - 同一organization_id内で同じdepartment_cdが存在しないか確認
2. **データ作成**
   - 新規レコードをINSERT
   - create_user_id, update_user_idに現在のユーザーIDを設定

バリデーション

- [ ] departmentCdが必須
- [ ] departmentCdが50文字以内
- [ ] departmentNameが必須
- [ ] departmentNameが200文字以内
- [ ] isActiveがboolean（指定時）

データベーステーブル

SQL例

```
-- 重複チェック
SELECT department_cd FROM department
WHERE organization_id = $1 AND department_cd = $2;

-- 新規作成
INSERT INTO department (
  organization_id, department_cd, department_name, is_active,
  create_user_id, create_at, update_user_id, update_at
) VALUES ($1, $2, $3, $4, $5, NOW(), $5, NOW())
RETURNING *;
```

セキュリティ

- [ ] organization_id必須
- [ ] SQLインジェクション対策

パフォーマンス

- トランザクション使用
- ユニーク制約でINSERT時の競合検知

---

6. 部署更新API

概要

既存の部署情報を更新します。

エンドポイント

```
PUT /api/v1/systemManagement/department/:departmentCd
```

HTTPメソッド

PUT

認証要件

- JWT認証必須
- ロール: System Admin または Data Admin
- テナント分離必須

ミドルウェアチェーン

```
router.put('/department/:departmentCd',
  authMiddleware,
  [
    param('departmentCd')
      .notEmpty()
      .withMessage('departmentCd is required')
      .isString()
      .withMessage('departmentCd must be a string'),
    body('departmentName')
      .optional()
      .isString()
      .withMessage('departmentName must be a string'),
    body('isActive')
      .optional()
      .isBoolean()
      .withMessage('isActive must be a boolean'),
  ],
  handleValidationErrors,
  departmentController.updateDepartment
);
```

リクエスト

パスパラメータ

パラメータ	型	必須	説明
departmentCd	string	○	部署CD

リクエストボディ

```
{
  "departmentName": "営業本部",
  "isActive": true
}
```

パラメータ	型	必須	説明
departmentName	string	×	部署名（200文字以内）
isActive	boolean	×	アクティブ状態

レスポンス

成功時（200 OK）

```
{
  "success": true,
  "data": {
    "departmentId": "DEPT001",
    "displayName": "営業本部",
    "isActive": true,
    "createdAt": "2026-01-01T00:00:00Z",
    "updatedAt": "2026-02-13T15:00:00Z"
  },
  "meta": {
    "timestamp": "2026-02-13T15:00:00Z",
    "requestId": "req_abc123"
  }
}
```

エラー時（404 Not Found）

```
{
  "success": false,
  "error": {
    "code": "DEPARTMENT_NOT_FOUND",
    "message": "Department not found"
  },
  "meta": {
    "timestamp": "2026-02-13T15:00:00Z",
    "requestId": "req_abc123"
  }
}
```

ビジネスロジック

1. **存在確認**
   - organization_idとdepartment_cdで部署を検索
   - 存在しない場合は404エラー
2. **データ更新**
   - 指定されたフィールドのみ更新
   - update_user_id, update_atを更新

バリデーション

- [ ] departmentCdが必須
- [ ] departmentNameが200文字以内（指定時）
- [ ] isActiveがboolean（指定時）

データベーステーブル

SQL例

```
UPDATE department
SET
  department_name = COALESCE($3, department_name),
  is_active = COALESCE($4, is_active),
  update_user_id = $5,
  update_at = NOW()
WHERE organization_id = $1 AND department_cd = $2
RETURNING *;
```

セキュリティ

- [ ] organization_id フィルタ必須
- [ ] SQLインジェクション対策

パフォーマンス

- プライマリキー更新（高速）
- トランザクション使用

---

7. 部署アクティブ状態更新API

概要

部署のアクティブ状態のみを更新します。

エンドポイント

```
PATCH /api/v1/systemManagement/department/:departmentCd/status
```

HTTPメソッド

PATCH

認証要件

- JWT認証必須
- ロール: System Admin または Data Admin
- テナント分離必須

ミドルウェアチェーン

```
router.patch('/department/:departmentCd/status',
  authMiddleware,
  [
    param('departmentCd')
      .notEmpty()
      .withMessage('departmentCd is required')
      .isString()
      .withMessage('departmentCd must be a string'),
  ],
  handleValidationErrors,
  departmentController.updateDepartmentStatus
);
```

リクエスト

パスパラメータ

パラメータ	型	必須	説明
departmentCd	string	○	部署CD

リクエストボディ

```
{
  "isActive": false
}
```

パラメータ	型	必須	説明
isActive	boolean	○	アクティブ状態

レスポンス

成功時（200 OK）


```
{
  "success": true,
  "data": {
    "departmentId": "DEPT001",
    "displayName": "営業部",
    "isActive": false,
    "createdAt": "2026-01-01T00:00:00Z",
    "updatedAt": "2026-02-13T15:00:00Z"
  },
  "meta": {
    "timestamp": "2026-02-13T15:00:00Z",
    "requestId": "req_abc123"
  }
}
```


エラー時（404 Not Found）


```
{
  "success": false,
  "error": {
    "code": "DEPARTMENT_NOT_FOUND",
    "message": "Department not found"
  },
  "meta": {
    "timestamp": "2026-02-13T15:00:00Z",
    "requestId": "req_abc123"
  }
}
```


ビジネスロジック

1. **存在確認**
    - organization_idとdepartment_cdで部署を検索
2. **ステータス更新**
    - is_activeフィールドのみ更新
    - update_user_id, update_atを更新

バリデーション

- [ ] departmentCdが必須
- [ ] isActiveが必須
- [ ] isActiveがboolean

データベーステーブル

SQL例

```
UPDATE department
SET
  is_active = $3,
  update_user_id = $4,
  update_at = NOW()
WHERE organization_id = $1 AND department_cd = $2
RETURNING *;
```

セキュリティ

- [ ] organization_id フィルタ必須
- [ ] SQLインジェクション対策

パフォーマンス

- プライマリキー更新（高速）

- トランザクション使用

🔗 関連ドキュメント

API設計書

- api_list.md - API一覧対応表
- api_role_management.md - ロール管理API
- api_master_management.md - マスタ管理API
- api_corporation_management.md - 会社管理API

規約・ガイドライン

- API設計ポリシー - API設計の統一ルール
- バックエンド規約 - 実装規約
- セキュリティ規約 - セキュリティ要件

参照ドキュメント

- データベース構造 - DB設計
- API構成定義 - API構成の参照ルール

📊 実装ファイルパス

ルートファイル

```
backend/src/routes/departmentRoutes.ts
```

コントローラーファイル

```
backend/src/controllers/departmentControler.ts
```

サービスファイル

```
backend/src/services/departmentService.ts
```

リポジトリファイル

```
backend/src/repositories/departmentRepository.ts
```

フロントエンドサービス

```
frontend/src/services/departmentApiService.ts
```

フロントエンドコンポーネント

```
frontend/src/components/SystemManagement/DepartmentManagement.tsx
```

✅ セキュリティチェックリスト

実装時に以下を確認してください：

認証・認可

- [ ] すべてのエンドポイントでJWT認証が必須
- [ ] authMiddlewareが適用されている
- [ ] System AdminまたはData Adminの権限が必要

マルチテナント分離

- [ ] すべてのDBクエリにorganization_idフィルタが含まれる
- [ ] クロステナントアクセスが不可能
- [ ] authMiddlewareでorganizationIdを取得

入力検証

- [ ] すべてのパラメータがバリデーションされている
- [ ] Parameterized Queriesを使用（SQLインジェクション対策）
- [ ] XSS対策（入力サニタイゼーション）
- [ ] 文字列長制限が強制されている

CSVアップロード対策

- [ ] ファイルサイズ制限（10MB）
- [ ] 許可されたMIMEタイプのみ（text/csv）
- [ ] ファイル内容のバリデーション
- [ ] CSVインジェクション対策

エラーハンドリング

- [ ] 統一されたエラーレスポンス形式
- [ ] エラーメッセージに機密情報が含まれない
- [ ] 適切なHTTPステータスコードを返す

ログ記録

- [ ] 重要な操作の監査証跡を記録
- [ ] エラー発生時の詳細ログ

📝 変更履歴

バージョン	日付	変更内容	担当者
1.0.0	2026-02-13	初版作成	Cline
		- 部署管理API 7エンドポイントの設計書作成	
		- 実装済み（100%）	

注意: この設計書は実装済みAPIの仕様を文書化したものです。実装の詳細については、実際のソースコード（backend/src/routes/departmentRoutes.ts等）を参照してください。